Name: **Shrey Pendurkar**
Class: **D15C**
Batch: **B**
Roll No: **38**

MLDL – Experiment 6

Aim

Apply K-Means and Hierarchical Clustering on sample datasets.

Dataset Source

- Dataset: Mall Customers Dataset
- Kaggle: https://www.kaggle.com/datasets/shwetabh123/mall-customers
- File used: Mall_Customers.csv

Dataset Description

The Mall Customers dataset contains demographic and behavioural information about customers of a shopping mall. It is designed for unsupervised customer segmentation tasks and has no target variable.

- Dataset size: 200 rows × 5 columns (no missing values)
- CustomerID (Integer): Unique identifier — excluded from clustering
- Genre (Categorical: Male/Female): Customer gender — excluded from clustering
- Age (Integer): Age in years
- Annual Income (k$) (Integer): Annual income in thousands of dollars
- Spending Score (1–100) (Integer): Score assigned by the mall based on customer behavior and spending patterns

Features used for clustering: Age, Annual Income (k$), Spending Score (1–100). The goal is to group customers into behaviorally meaningful segments without any predefined labels.

Experiment 1: K-Means Clustering

1. Mathematical Formulation of the Algorithm

Let the dataset be $X = \{x_1, x_2, \ldots, x_n\}$, with each $x_i \in \mathbb{R}^d$. K-Means partitions the data into K disjoint clusters $C_1, C_2, \ldots, C_K$ by minimising the Within-Cluster Sum of Squares (WCSS).

Objective Function (WCSS):

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k is the centroid of cluster C_k.

Centroid Update:

$$\mu_k = (1 / |C_k|) \sum_{x_i \in C_k} x_i$$

Algorithm Steps:

- Step 1 — Initialise: Choose K centroids using K-Means++ (spreads initial centroids across the data to reduce bad initialisation risk)
- Step 2 — Assign: Assign each sample to its nearest centroid using Euclidean distance: $\text{argmin}_k \|x_i - \mu_k\|$
- Step 3 — Update: Recompute each centroid μ_k as the mean of all samples assigned to cluster C_k
- Step 4 — Repeat Steps 2–3 until convergence (centroids stop changing or max iterations reached)

---

2. Algorithm Limitations

1. Requires Predefined K

The number of clusters must be specified before training. Choosing K incorrectly causes either merging of distinct groups (K too small) or fragmentation of natural groups (K too large). Methods like the silhouette score or elbow plot are used to guide K selection empirically.

2. Sensitive to Initial Centroid Placement

K-Means converges to local minima, so different random initialisations can yield different final cluster assignments. K-Means++ reduces this risk by spreading initial centroids but cannot eliminate it entirely. Multiple runs (n_init) mitigate this.

3. Assumes Spherical, Similar-Sized Clusters

K-Means uses Euclidean distance and mean centroids, making it well-suited only for compact, roughly equal-sized clusters. It performs poorly on elongated, irregularly shaped, or varying-density clusters.

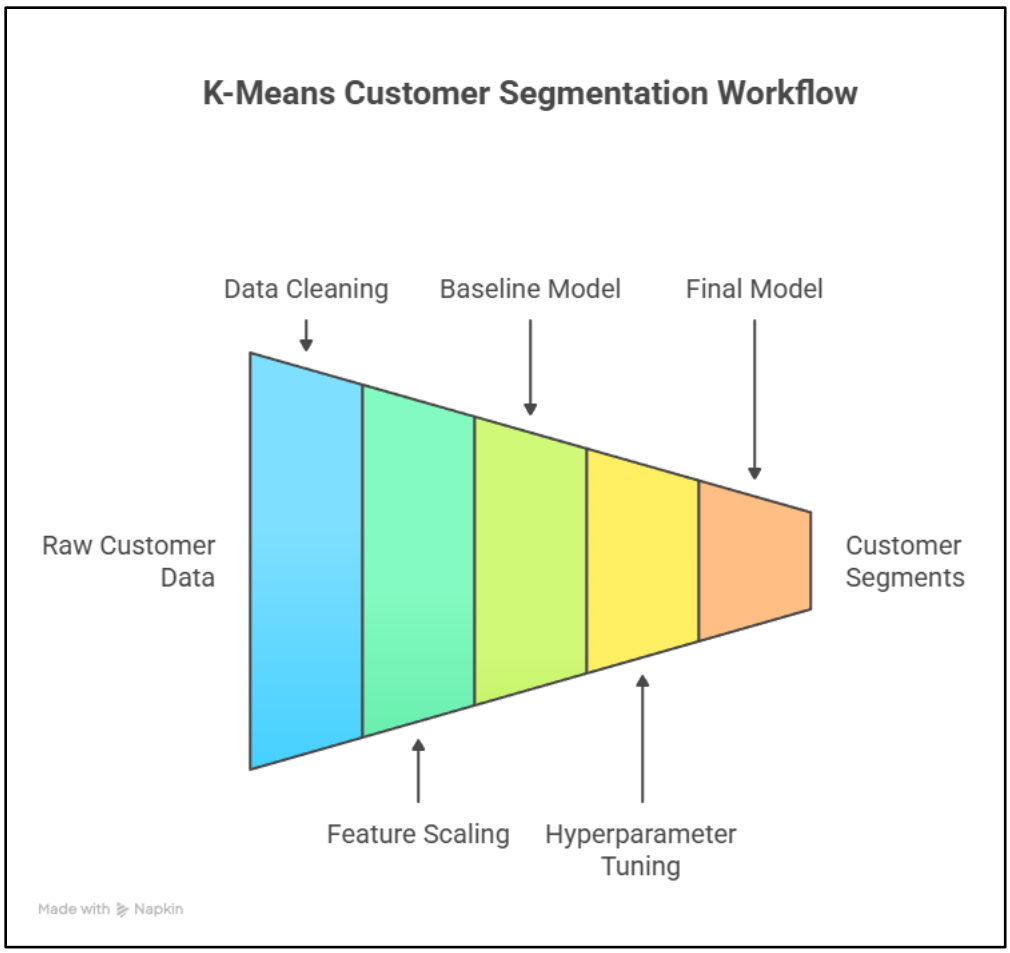
4. Not Robust to Outliers

Outliers can significantly shift centroids, distorting cluster boundaries. Unlike density-based methods (e.g., DBSCAN), K-Means has no built-in mechanism for handling outlier points.

---

3. Methodology / Workflow

1. **Data Collection**
   - Download Mall_Customers.csv from Kaggle and load into a Pandas DataFrame.
2. **Data Cleaning and Preparation**
   - Check for missing values (none present). Drop any incomplete rows if found.
   - Keep only Age, Annual Income (k$), and Spending Score (1–100) for clustering; exclude CustomerID and Genre.
3. **Feature Scaling**
   - Apply StandardScaler to standardise all three features to zero mean and unit variance, ensuring equal contribution to Euclidean distance.
4. **Baseline Model (Without Tuning)**
   - Train KMeans with K=2 (minimum sensible default), init='k-means++', n_init=10, random_state=42.
   - Evaluate: silhouette score, WCSS, cluster sizes, and cluster feature means.
5. **Hyperparameter Tuning (K selection)**
   - Train K-Means for K = 2 to 10. For each K, compute the silhouette score.
   - Select the K with the highest silhouette score as the optimal number of clusters. Also inspect the WCSS elbow plot.
6. **Final Model Training and Cluster Assignment**
   - Train a final KMeans model with the best K, using n_init=30 for stability. Assign cluster labels to all customers.
7. **Cluster Profiling and Visualisation**
   - Compute cluster-wise mean Age, Income, and Spending Score. Interpret each cluster as a customer segment.
   - Visualise using PCA 2D scatter plot, cluster size bar chart, and feature means heatmap.



4. Performance Analysis

Model: KMeans(n_clusters=2, init='k-means++', n_init=10, random_state=42)

Features: Age, Annual Income (k$), Spending Score (1–100) | Scaled with StandardScaler

Metric	Value
K (Clusters)	2
Silhouette Score	0.3355
WCSS (Inertia)	389.39

Cluster	Age (mean)	Annual Income (k$)	Spending Score (1–100)
Cluster 0 (n=103)	48.73	60.50	32.18
Cluster 1 (n=97)	28.36	60.63	69.33

With only K=2, the algorithm identifies one broad age-based split: older customers with low spending vs younger customers with high spending. Both clusters have nearly identical mean income (~60k), confirming that at K=2 the algorithm cannot distinguish by income level at all. The silhouette score of 0.3355 indicates moderate cluster quality — clusters exist but have significant overlap. This represents the baseline before tuning.

---

5. Hyperparameter Tuning

Tuning Strategy: Silhouette Score Sweep (K = 2 to 10)

K (Clusters)	Silhouette Score
2	0.3355
3	0.3578
4	0.4040
5	0.4166
6	0.4274 ← Best
7	0.4172
8	0.4087
9	0.4201
10	0.3973

Optimal K = 6 | Best Silhouette Score: 0.4274 | WCSS at K=6: 133.87

Final Model (K=6) — Cluster Feature Means:

Cluster	Age (mean)	Annual Income (k$)	Spending Score (1–100)
Cluster 0 (n=33)	41.94	88.94	16.97
Cluster 1 (n=45)	56.33	54.27	49.07
Cluster 2 (n=24)	25.25	25.83	76.92
Cluster 3 (n=39)	32.69	86.54	82.13
Cluster 4 (n=38)	26.68	57.58	47.79
Cluster 5 (n=21)	45.52	26.29	19.38

Cluster Interpretations:

- Cluster 3 — Young/mid-age, high-income, high-spending (Premium): Income ~86.5k, Spending ~82.1. Top-value segment with strongest revenue potential.

- Cluster 2 — Very young, low-income, high-spending (Aspirational): Income ~25.8k, Spending ~76.9. Spend aggressively relative to income; ideal for loyalty programmes.
- Cluster 1 — Older, mid-income, moderate-spending (Stable): Age ~56, balanced income and spending. Consistent but not extreme; respond differently to promotions.
- Cluster 4 — Young, mid-income, moderate-spending (Growth potential): Age ~27, moderate across all metrics. Strong upsell opportunity with targeted campaigns.
- Cluster 0 — Middle-aged, high-income, low-spending (Under-utilised): Income ~88.9k, Spending only ~17. High strategic value — increasing engagement can greatly boost revenue.
- Cluster 5 — Middle-aged, low-income, low-spending (Low priority): Income ~26.3k, Spending ~19.4. Heavy marketing investment unlikely to yield proportional returns.

Key Observations:

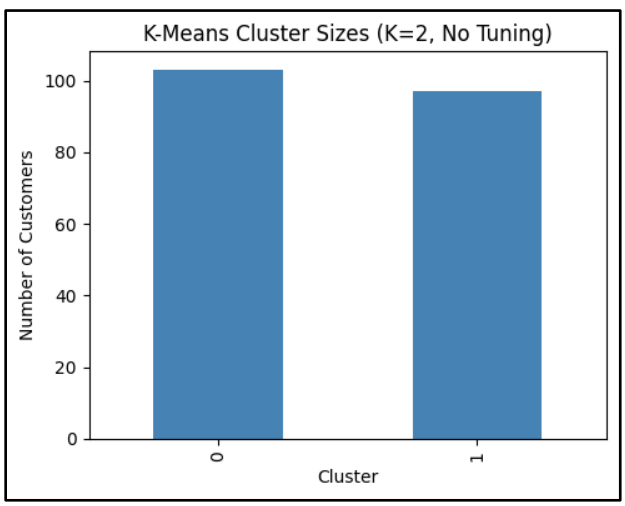
- Silhouette score improved from 0.3355 (K=2) → 0.4274 (K=6), a gain of +0.0919, indicating substantially tighter and better-separated clusters
- WCSS dropped from 389.39 → 133.87, confirming much greater intra-cluster cohesion with 6 clusters
- K=6 uncovers income-based differentiation completely invisible at K=2: clusters 0 and 3 (high-income) vs clusters 2 and 5 (low-income) now separate clearly
- The combination of high-income–low-spending (Cluster 0) and high-income–high-spending (Cluster 3) demonstrates that K-Means distinguishes not just by wealth but by spending behaviour

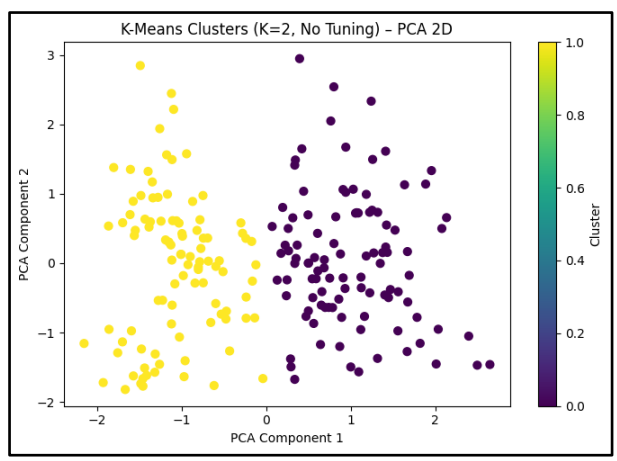
---

6. Code and Output – Without Tuning

```
# K-Means Clustering – Without Tuning (Default K=2)
# Dataset: Mall Customers (Mall_Customers.csv)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# Load dataset
df = pd.read_csv("Mall_Customers.csv")
print("Shape:", df.shape)
print(df.head())
print("\nMissing values:\n", df.isnull().sum())
df = df.dropna()

# Feature selection
features = ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
X = df[features]
```

```
# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# K-Means with default K=2 (no tuning)
kmeans = KMeans(n_clusters=2, init="k-means++", n_init=10,
                max_iter=300, random_state=42)
labels = kmeans.fit_predict(X_scaled)
df["KMeansCluster"] = labels

# Silhouette score
sil = silhouette_score(X_scaled, labels)

print("\n========== K-Means (No Tuning, K=2) ==========")
print(f"K = 2")
print(f"Silhouette Score: {sil:.4f}")
print(f"Inertia (WCSS): {kmeans.inertia_:.4f}")

# Cluster sizes
print("\nCluster Sizes:")
print(df["KMeansCluster"].value_counts().sort_index())

# Cluster profiling
summary = df.groupby("KMeansCluster")[features].mean()
print("\nCluster Feature Means:")
print(summary.round(2))

# PCA visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(7, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap="viridis", s=40)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("K-Means Clusters (K=2, No Tuning) – PCA 2D")
plt.colorbar(label="Cluster")
plt.tight_layout()
plt.show()

# Cluster size bar chart
df["KMeansCluster"].value_counts().sort_index().plot(kind="bar", figsize=(5, 4), color="steelblue")
plt.xlabel("Cluster")
plt.ylabel("Number of Customers")
plt.title("K-Means Cluster Sizes (K=2, No Tuning)")
plt.tight_layout()
plt.show()
```

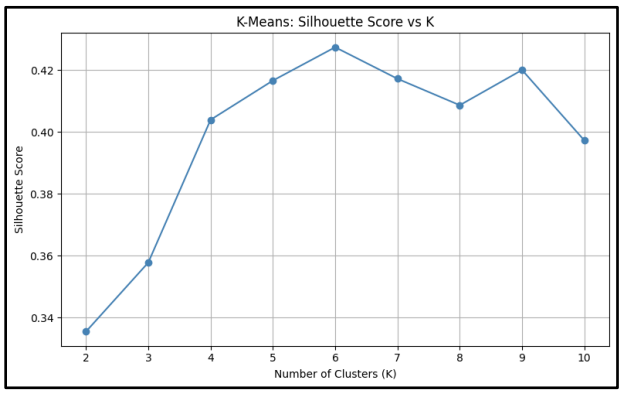


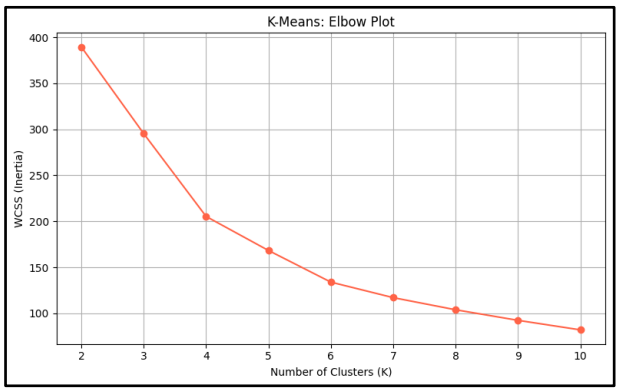


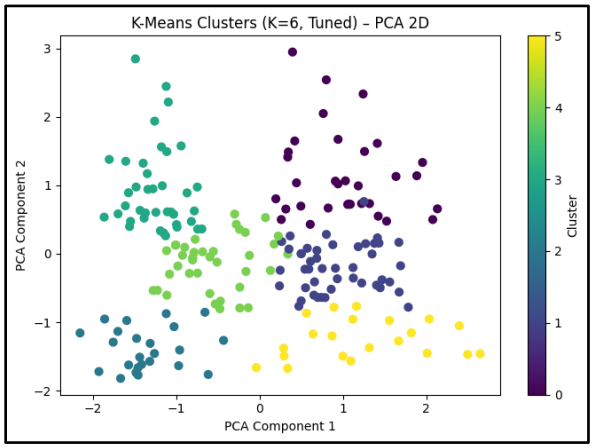
7. Code and Output – With Tuning

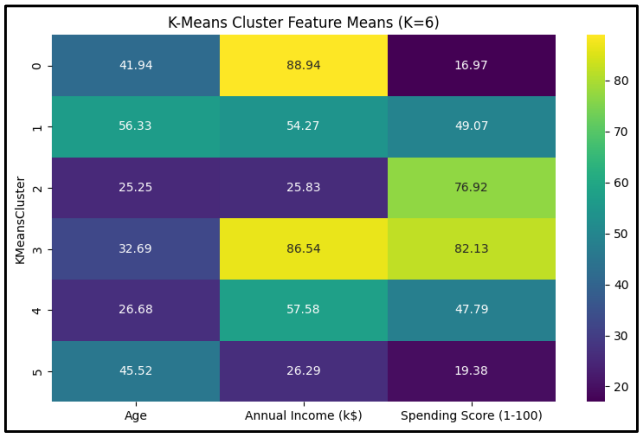
```
# K-Means Clustering – With Tuning
(Silhouette-Optimized K)
# Dataset: Mall Customers
(Mall_Customers.csv)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import
StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import
silhouette_score
from sklearn.decomposition import PCA

# Load and prepare dataset
df = pd.read_csv("Mall_Customers.csv")
df = df.dropna()
features = ["Age", "Annual Income (k$)",
"Spending Score (1-100)"]
X = df[features]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Sweep K=2 to 10, collect silhouette
scores and WCSS
k_values = range(2, 11)
sil_scores = []
wcss = []

for k in k_values:
    km = KMeans(n_clusters=k, init="k-
means++", n_init=20,
                max_iter=300,
random_state=42)
    lbl = km.fit_predict(X_scaled)
    sil_scores.append(silhouette_score(X
_scaled, lbl))
    wcss.append(km.inertia_)

# Print all scores
print("K\tSilhouette\tWCSS")
for k, s, w in zip(k_values, sil_scores,
wcss):
    print(f"{k}\t{s:.4f}\t\t{w:.2f}")

best_k =
list(k_values)[np.argmax(sil_scores)]
print(f"\nOptimal K (highest silhouette):
{best_k}")
print(f"Best Silhouette Score:
{max(sil_scores):.4f}")

# Silhouette score plot
plt.figure(figsize=(8, 5))
plt.plot(list(k_values), sil_scores,
marker="o", color="steelblue")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.title("K-Means: Silhouette Score vs
K")
plt.grid(True)
plt.tight_layout()
plt.show()

# Elbow (WCSS) plot
plt.figure(figsize=(8, 5))
plt.plot(list(k_values), wcss, marker="o",
color="tomato")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS (Inertia)")
plt.title("K-Means: Elbow Plot")
plt.grid(True)
plt.tight_layout()
plt.show()

# Final model with best K
kmeans_final =
KMeans(n_clusters=best_k, init="k-
means++", n_init=30,
                max_iter=300,
random_state=42)
df["KMeansCluster"] =
kmeans_final.fit_predict(X_scaled)

final_sil = silhouette_score(X_scaled,
df["KMeansCluster"])
print(f"\n========== K-Means (Tuned,
K={best_k}) ==========")
print(f"Silhouette Score: {final_sil:.4f}")
```

```
print(f"Inertia (WCSS): {kmeans_final.inertia_:.4f}")

# Cluster sizes
print("\nCluster Sizes:")
print(df["KMeansCluster"].value_counts().sort_index())

# Cluster profiling
summary = df.groupby("KMeansCluster")[features].mean()
print("\nCluster Feature Means:")
print(summary.round(2))

# PCA visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(7, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1],
            c=df["KMeansCluster"], cmap="viridis", s=40)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title(f"K-Means Clusters (K={best_k}, Tuned) – PCA 2D")
plt.colorbar(label="Cluster")
plt.tight_layout()
plt.show()

# Heatmap of cluster feature means
plt.figure(figsize=(8, 5))
sns.heatmap(summary, annot=True, fmt=".2f", cmap="viridis")
plt.title(f"K-Means Cluster Feature Means (K={best_k})")
plt.tight_layout()
plt.show()
```









Experiment 2: Hierarchical Clustering

1. Mathematical Formulation of the Algorithm

Hierarchical Agglomerative Clustering (HAC) builds a hierarchy of clusters bottom-up. Initially each data point is its own cluster. At each step, the two most similar clusters are merged according to a linkage criterion until all points form a single cluster.

Distance Metric (Euclidean):

$$d(x_i, x_x) = ||x_i - x_x||_2 = \sqrt{(\Sigma_j (x_{ij} - x_{xj})^2)}$$

Ward Linkage Criterion:

At each step, merge the pair of clusters (A, B) that results in the minimum increase in total within-cluster variance:

$$\Delta(A, B) = (|A| \cdot |B|) / (|A| + |B|) \cdot ||m_A - m_B||^2$$

where m_A and m_B are the centroids of clusters A and B, and |A|, |B| are their sizes. Ward linkage tends to produce compact, roughly spherical clusters.

Dendrogram:

The merging process produces a dendrogram — a tree diagram where the height of each merge represents the distance at which clusters were joined. Cutting the dendrogram at a chosen height yields K clusters. A large vertical gap before a merge suggests a natural cluster boundary.

---

2. Algorithm Limitations

1. High Computational and Memory Cost

Time complexity is typically $O(n^2 \log n)$ or worse. The algorithm often stores an n×n distance matrix, making it impractical for very large datasets (thousands of samples).

2. Irreversible Merges

Once two clusters are merged, they are never split. Early erroneous merges — caused by noise or outliers — cannot be corrected and propagate through the entire hierarchy.

3. Sensitivity to Noise and Outliers

Outliers may form their own singleton branches or force incorrect merges, distorting the dendrogram and the resulting cluster structure.

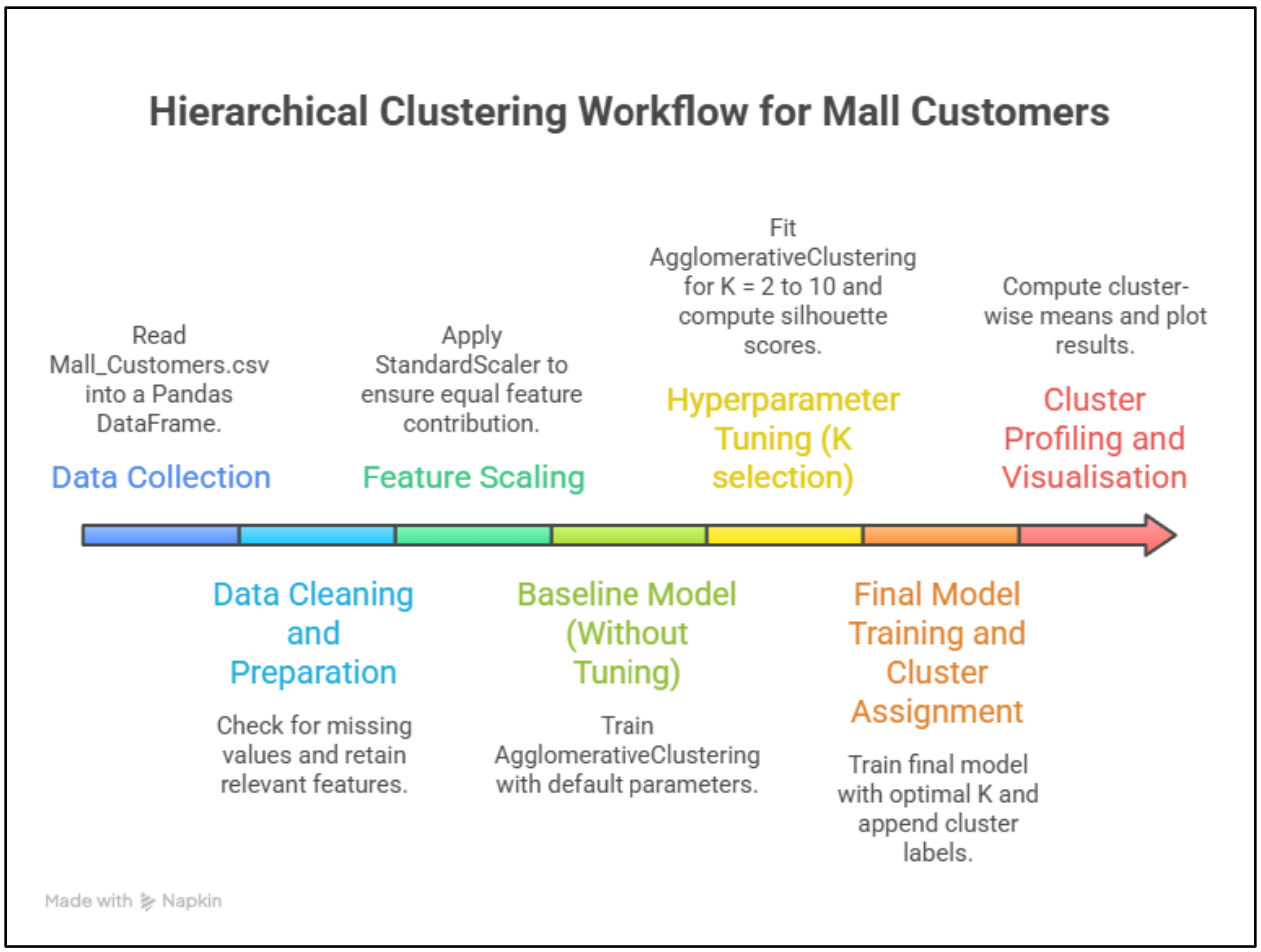
4. Dependence on Distance Metric and Linkage Method

Different linkage choices (single, complete, average, Ward) and distance metrics (Euclidean, Manhattan, cosine) can produce very different cluster structures on the same data, requiring domain knowledge or empirical evaluation to select appropriately.

---

3. Methodology / Workflow

1. **Data Collection**
   - Read Mall_Customers.csv into a Pandas DataFrame.
2. **Data Cleaning and Preparation**
   - Check for missing values (none present). Drop incomplete rows if found.
   - Retain only Age, Annual Income (k$), and Spending Score (1–100).
3. **Feature Scaling**
   - Apply StandardScaler to ensure all features contribute equally to Euclidean distance.
4. **Baseline Model (Without Tuning)**
   - Train AgglomerativeClustering with n_clusters=2 and linkage='ward'.
   - Evaluate: silhouette score, cluster sizes, and cluster feature means.
5. **Hyperparameter Tuning (K selection)**
   - Fit AgglomerativeClustering for K = 2 to 10. Compute silhouette score for each K.
   - Select the K with the highest silhouette score. Additionally verify using the dendrogram — a large height gap before a merge supports the chosen K.
6. **Final Model Training and Cluster Assignment**
   - Train final AgglomerativeClustering model with optimal K and Ward linkage. Append cluster labels as HierCluster column.
7. **Cluster Profiling and Visualisation**
   - Compute cluster-wise mean Age, Income, and Spending Score. Interpret each cluster.
   - Plot: dendrogram (first 100 samples), PCA 2D scatter, cluster size bar chart, feature means heatmap.



4. Performance Analysis

Model: AgglomerativeClustering(n_clusters=2, linkage='ward')

Features: Age, Annual Income (k$), Spending Score (1–100) | Scaled with StandardScaler

Metric	Value
K (Clusters)	2
Silhouette Score	0.3180

Cluster	Age (mean)	Annual Income (k$)	Spending Score (1–100)
Cluster 0 (n=105)	28.84	61.91	66.20
Cluster 1 (n=95)	49.92	59.06	32.52

At K=2, Ward linkage produces the same broad age-spending split as K-Means: a younger/high-spending cluster vs an older/low-spending cluster, both with similar mid-range incomes (~60–62k). The silhouette score of 0.3180 is slightly lower than K-Means at K=2 (0.3355), reflecting that agglomerative merging at this coarse level is marginally less cohesive than centroid-based partitioning. Income-based differentiation is completely absent at this level of granularity.

---

5. Hyperparameter Tuning

Tuning Strategy: Silhouette Score Sweep (K = 2 to 10)

K (Clusters)	Silhouette Score
2	0.3180
3	0.3215
4	0.3615
5	0.3900
6	0.4201 ← Best
7	0.3983
8	0.3665
9	0.3754
10	0.3809

Optimal K = 6 | Best Silhouette Score: 0.4201

Final Model (K=6) — Cluster Feature Means:

Cluster	Age (mean)	Annual Income (k$)	Spending Score (1–100)
Cluster 0 (n=45)	27.38	57.51	45.84
Cluster 1 (n=45)	56.40	55.29	48.36
Cluster 2 (n=39)	32.69	86.54	82.13
Cluster 3 (n=28)	43.89	91.29	16.68
Cluster 4 (n=22)	44.32	25.77	20.27
Cluster 5 (n=21)	24.81	25.62	80.24

Cluster Interpretations:

- Cluster 2 — Young/mid-age, high-income, high-spending (Premium): Income ~86.5k, Spending ~82.1. Top-value customers; ideal for premium and loyalty offers.

- Cluster 5 — Very young, low-income, high-spending (Aspirational): Income ~25.6k, Spending ~80.2. Trend-driven young shoppers; attractive for promotions and long-term relationship building.
- Cluster 1 — Older, mid-income, moderate-spending (Stable): Age ~56, balanced income and spending. Conservative, consistent segment.
- Cluster 0 — Young, mid-income, moderate-spending (Growth potential): Age ~27, mid income and spending. Can be grown into higher-value behavior with targeted campaigns.
- Cluster 3 — Middle-aged, very high-income, very low-spending (Under-utilised): Highest income (~91.3k) but spending only ~16.7. Strategically critical — even small engagement increases yield large revenue gains.
- Cluster 4 — Middle-aged, low-income, low-spending (Low priority): Income ~25.8k, Spending ~20.3. Lower-value segment; heavy marketing spend unlikely to yield proportional returns.

Key Observations:

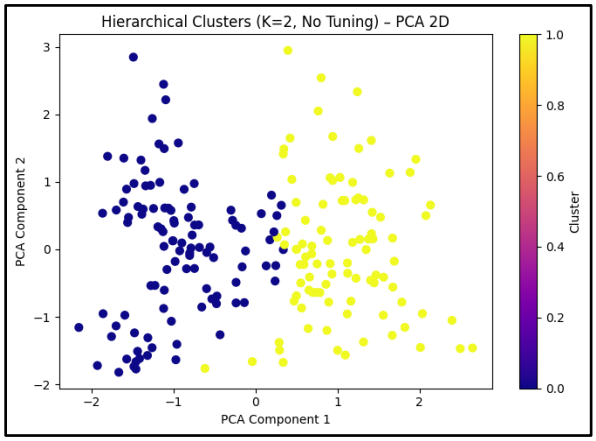
- Silhouette score improved from 0.3180 (K=2) → 0.4201 (K=6), a gain of +0.1021
- K=6 independently identifies the same six behavioural archetypes as K-Means, validating the cluster structure across two different algorithmic approaches
- Cluster 3 (highest income ~91.3k, lowest spending ~16.7) is the most strategically important finding — under-engaged wealthy customers represent maximum untapped revenue
- Unlike K-Means, Ward linkage produces two equal-sized moderate clusters (Clusters 0 and 1, both n=45) split by age rather than spending, reflecting the hierarchy's bottom-up merging strategy
- The dendrogram would show a pronounced height gap before the final merge from 6 to 5 clusters, providing independent structural confirmation that K=6 is a natural cut point

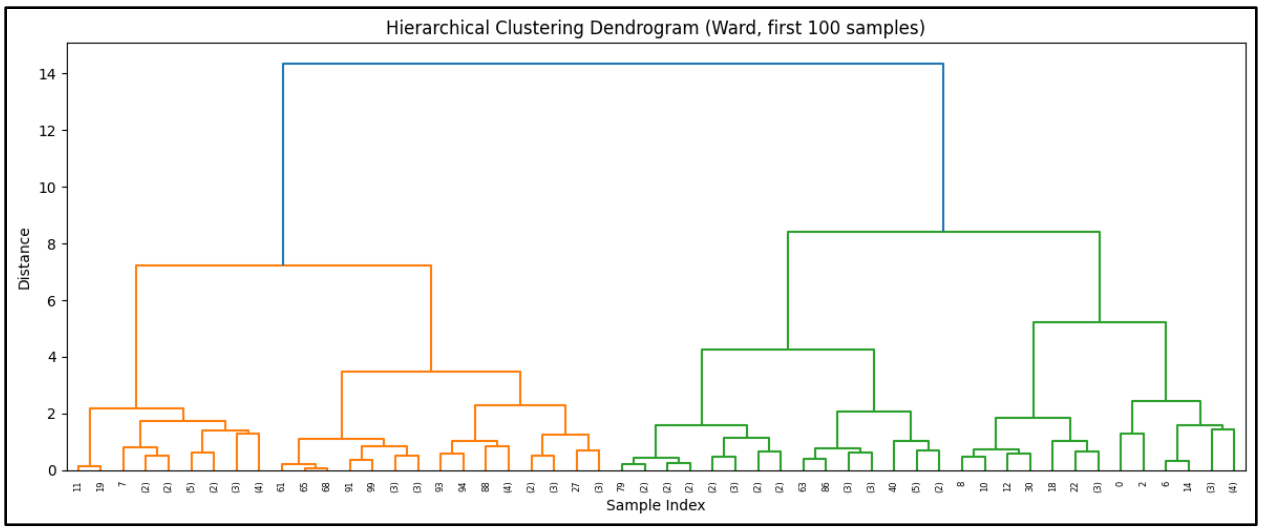
---

6. Code and Output – Without Tuning

```
# Hierarchical (Agglomerative) Clustering – Without Tuning (K=2)
# Dataset: Mall Customers (Mall_Customers.csv)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import dendrogram, linkage

# Load dataset
df = pd.read_csv("Mall_Customers.csv")
df = df.dropna()
```

```
features = ["Age", "Annual Income (k$)",
"Spending Score (1-100)"]
X = df[features]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Agglomerative Clustering with K=2 (no
tuning)
model =
AgglomerativeClustering(n_clusters=2,
linkage="ward")
labels = model.fit_predict(X_scaled)
df["HierCluster"] = labels

# Silhouette score
sil = silhouette_score(X_scaled, labels)

print("\n========== Hierarchical
Clustering (No Tuning, K=2)
==========")
print(f"K = 2")
print(f"Silhouette Score: {sil:.4f}")

# Cluster sizes
print("\nCluster Sizes:")
print(df["HierCluster"].value_counts().sor
t_index())

# Cluster profiling
summary =
df.groupby("HierCluster")[features].mea
n()
print("\nCluster Feature Means:")
print(summary.round(2))

# PCA visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(7, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1],
c=labels, cmap="plasma", s=40)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("Hierarchical Clusters (K=2, No
Tuning) – PCA 2D")
plt.colorbar(label="Cluster")
plt.tight_layout()
plt.show()

# Dendrogram (first 100 samples for
readability)
linked = linkage(X_scaled[:100],
method="ward")
plt.figure(figsize=(12, 5))
dendrogram(linked,
truncate_mode="level", p=5)
plt.title("Hierarchical Clustering
Dendrogram (Ward, first 100 samples)")
plt.xlabel("Sample Index")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
```

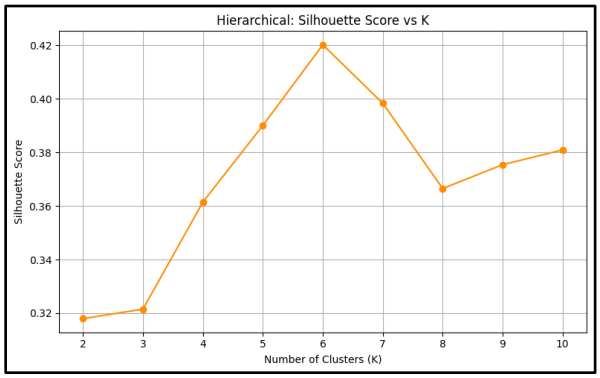


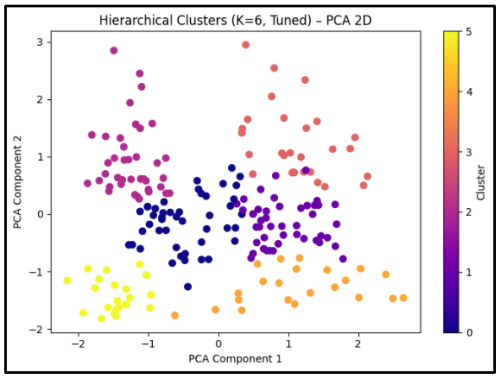


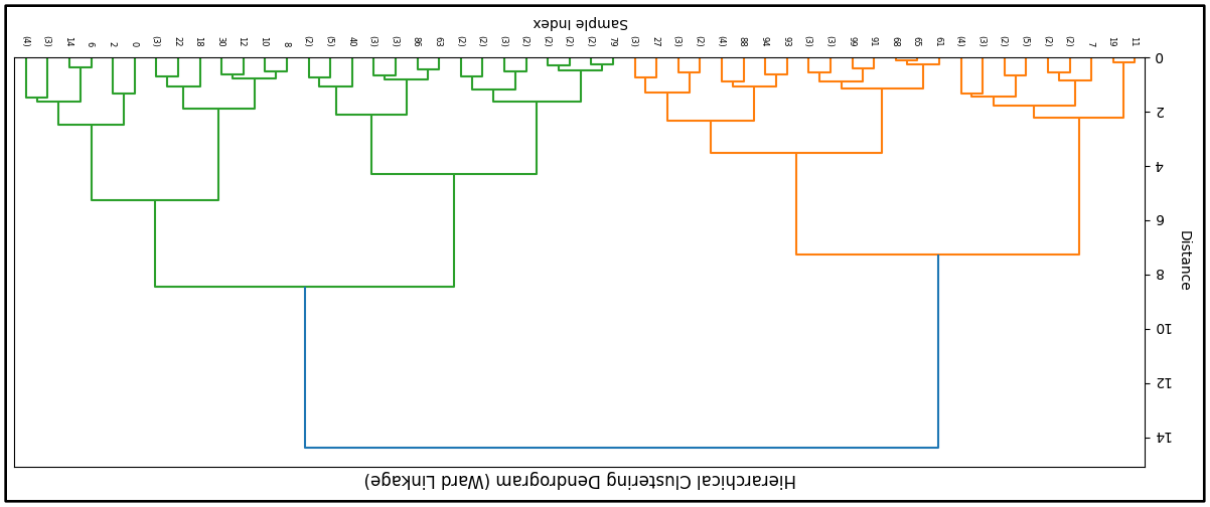
7. Code and Output – With Tuning

```
# Hierarchical (Agglomerative) Clustering – With Tuning (Silhouette-Optimized K)
# Dataset: Mall Customers (Mall_Customers.csv)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import dendrogram, linkage

# Load and prepare dataset
df = pd.read_csv("Mall_Customers.csv")
df = df.dropna()
features = ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
X = df[features]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Sweep K=2 to 10, collect silhouette scores
k_values = range(2, 11)
sil_scores = []

for k in k_values:
    model = AgglomerativeClustering(n_clusters=k, linkage="ward")
    lbl = model.fit_predict(X_scaled)
    sil_scores.append(silhouette_score(X_scaled, lbl))

# Print all scores
print("K\tSilhouette Score")
for k, s in zip(k_values, sil_scores):
    print(f"{k}\t{s:.4f}")

best_k = list(k_values)[np.argmax(sil_scores)]
print(f"\nOptimal K (highest silhouette): {best_k}")
print(f"Best Silhouette Score: {max(sil_scores):.4f}")

# Silhouette score plot
plt.figure(figsize=(8, 5))
plt.plot(list(k_values), sil_scores, marker="o", color="darkorange")
plt.xlabel("Number of Clusters (K)")
```

```
plt.ylabel("Silhouette Score")
plt.title("Hierarchical: Silhouette Score
vs K")
plt.grid(True)
plt.tight_layout()
plt.show()

# Final model with best K
hier_final =
AgglomerativeClustering(n_clusters=bes
t_k, linkage="ward")
df["HierCluster"] =
hier_final.fit_predict(X_scaled)

final_sil = silhouette_score(X_scaled,
df["HierCluster"])
print(f"\n========== Hierarchical
Clustering (Tuned, K={best_k})
==========")
print(f"Silhouette Score: {final_sil:.4f}")

# Cluster sizes
print("\nCluster Sizes:")
print(df["HierCluster"].value_counts().sor
t_index())

# Cluster profiling
summary =
df.groupby("HierCluster")[features].mea
n()
print("\nCluster Feature Means:")
print(summary.round(2))

# PCA visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(7, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1],
            c=df["HierCluster"],
cmap="plasma", s=40)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title(f"Hierarchical Clusters
(K={best_k}, Tuned) – PCA 2D")
plt.colorbar(label="Cluster")
plt.tight_layout()
plt.show()

# Heatmap of cluster feature means
plt.figure(figsize=(8, 5))
sns.heatmap(summary, annot=True,
fmt=".2f", cmap="plasma")
plt.title(f"Hierarchical Cluster Feature
Means (K={best_k})")
plt.tight_layout()
plt.show()

# Dendrogram (first 100 samples)
linked = linkage(X_scaled[:100],
method="ward")
plt.figure(figsize=(12, 5))
dendrogram(linked,
truncate_mode="level", p=5)
plt.title("Hierarchical Clustering
Dendrogram (Ward Linkage)")
plt.xlabel("Sample Index")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
```







Conclusion

Both K-Means and Hierarchical Agglomerative Clustering (Ward linkage) consistently identify **K = 6** as the optimal number of customer segments for the Mall Customers dataset, supported by strong silhouette scores (0.4274 and 0.4201). This agreement across two different clustering approaches reinforces the reliability of the six-segment structure. In contrast, the baseline model (K = 2) is overly simplistic, capturing only broad age–spending differences while ignoring income variation and masking meaningful patterns.

With K = 6, a much richer and actionable segmentation emerges: high-income customers split into premium buyers and under-engaged individuals, low-income groups divided by spending behaviour, and mid-income customers differentiated by age profiles. While both methods uncover similar segment archetypes, minor differences in cluster boundaries arise due to their underlying mechanisms (centroid-based vs. hierarchical merging). Overall, the results demonstrate that silhouette-guided selection of K, validated across multiple algorithms, is essential for deriving meaningful and business-relevant customer insights.